

12-02 — Migrations and Schema

Section

12 — Query Builder, Migrations, and Transactions in TypeORM

Subtopic

Migrations and Schema

Duration

~2–2.5 hours

Tags

Theory

Practical

TypeScript

Topics covered

Schema synchronisation vs migrations, creating migrations manually, generating migrations from entities, running migrations, reverting migrations, migrations table, production workflow

Learning Objectives — This Subtopic

- Explain the difference between `synchronize: true` and migration-based schema management, and when each is appropriate.
- Create a migration manually using the TypeORM CLI and implement `up()` and `down()` methods.
- Generate a migration automatically from entity changes using `typeorm migration:generate`.
- Run pending migrations and revert the most recent migration against a live database.
- Describe how TypeORM tracks applied migrations through the `migrations` table.
- Structure a project's DataSource configuration to support a reliable migration workflow in production.

Why Migrations Exist

In Section 10 you configured TypeORM's `DataSource` with `synchronize: true`. That setting instructs TypeORM to compare your entity definitions against the live schema on every application start and issue the necessary `ALTER TABLE` / `CREATE TABLE` statements automatically. For development iteration this is fast and convenient.

In production, it is dangerous. A mistaken column rename, a changed nullable constraint, or a dropped relation can silently delete data or break a running service. The schema needs to change in a controlled, auditable, reversible way — which is exactly what **migrations** provide.

A migration is a TypeScript file containing two methods:

- `up()` — applies the change (e.g., adds a column).
- `down()` — reverts the change (e.g., drops that column).

TypeORM maintains a table called `migrations` in your database. Each time a migration is executed, its name is recorded there. The CLI only runs migrations that do not yet appear in that table, preventing double application.

Analogy: Think of migrations as a version-controlled changelog for your database schema — the same way Git tracks changes to source code. Each migration is a commit: it has a description, an author (implied by when it was generated), and it can be undone. Running `migration:run` is equivalent to `git pull`; running `migration:revert` is `git revert`.

Schema Synchronisation vs Migrations

These two approaches are mutually exclusive in practice. The table below summarises when to use each:

Aspect	<code>synchronize: true</code>	Migrations
Speed of iteration	Instant — no files to generate	Slower — requires generating and running files

Aspect	<code>synchronize: true</code>	Migrations
Data safety	None — destructive changes happen silently	Explicit — every change is reviewed before running
Reversibility	None — no <code>down()</code> available	Full — <code>migration:revert</code> undoes the last batch
Team collaboration	Problematic — each dev's schema may diverge	Reliable — migration files are committed to Git
Recommended for	Local development only	Staging, production, or any shared environment

Important: Never deploy an application to production with `synchronize: true`. TypeORM may drop columns it believes are no longer mapped, even if those columns hold live data. The *only* safe schema-change mechanism in production is migrations.

The standard pattern is to maintain two DataSource configurations — one for the running application (synchronize off, migrations enabled) and one shorthand entry point used exclusively by the TypeORM CLI.

Project Setup for Migrations

We will extend the project from Sections 10 and 11. The examples below use a `User` entity whose schema evolves across multiple migrations.

```
project-root/
├─ src/
│   └─ entity/
│       └─ User.ts
│   └─ migration/      <-- TypeORM writes generated files here
│   └─ data-source.ts  <-- shared DataSource (used by app AND CLI)
```

```
|   └─ index.ts  
└─ package.json  
└─ tsconfig.json
```

DataSource Configuration

The DataSource must be exported from a dedicated file so the TypeORM CLI can import it. The two key migration-related options are `migrations` (a glob pointing to compiled migration files) and

`migrationsTableName` (defaults to `"migrations"`).

```
src/data-source.ts
```

```
import "reflect-metadata";  
import { DataSource } from "typeorm";  
import { User } from "../entity/User";  
  
export const AppDataSource = new DataSource({  
  type: "postgres",  
  host: process.env.DB_HOST ?? "localhost",  
  port: Number(process.env.DB_PORT ?? 5432),  
  username: process.env.DB_USER ?? "postgres",  
  password: process.env.DB_PASS ?? "password",  
  database: process.env.DB_NAME ?? "appdb",  
  
  // Entities are loaded from their compiled JS output  
  entities: [User],  
  
  // NEVER true in production  
  synchronize: false,  
  
  // Compiled migration files to load and run  
  migrations: ["dist/migration/*.js"],  
  
  // Optional: keep the migrations table name explicit  
  migrationsTableName: "migrations",  
  
  logging: ["migration"],  
});
```

Key Insight: The `migrations` glob points to the *compiled* `.js` files in `dist/`, not the TypeScript sources. The CLI compiles via your `tsconfig.json` before running. Make sure `outDir` in `tsconfig.json` is set to `"dist"`.

CLI Script Entry in package.json

The TypeORM CLI needs to know which DataSource to use. Pass it via the `-d` flag. Add these scripts to `package.json` to avoid typing the full path each time:

package.json (scripts section)

```
{
  "scripts": {
    "build": "tsc",
    "start": "node dist/index.js",
    "typeorm": "typeorm-ts-node-commonjs -d src/data-source.ts"
  }
}
```

Note: `typeorm-ts-node-commonjs` is a CLI wrapper that runs TypeORM without a separate `tsc` step, using `ts-node` under the hood. Install it with `npm install --save-dev typeorm ts-node`. For ESM projects use `typeorm-ts-node-esm` instead.

Creating a Migration Manually

Manual migrations are used when you need precise control over SQL — for example, backfilling data, adding a partial index, or making a change that cannot be inferred from entity decorators alone.

The `migration:create` command generates an empty migration file with timestamped name:

```
npm run typeorm migration:create src/migration/CreateUserTable
```

Output:

Migration /path/to/project/src/migration/1748000000000-CreateUserTable.ts has been created successfully.

The generated file contains the skeleton you fill in:

```
src/migration/1748000000000-CreateUserTable.ts
```

```
import { MigrationInterface, QueryRunner } from "typeorm";

export class CreateUserTable1748000000000 implements MigrationInterface {

    public async up(queryRunner: QueryRunner): Promise<void> {
        // Write the schema change here
    }

    public async down(queryRunner: QueryRunner): Promise<void> {
        // Write the revert here
    }
}
```

Fill in the `up()` and `down()` methods using `QueryRunner`'s schema helpers:

```
src/migration/1748000000000-CreateUserTable.ts
```

```
import { MigrationInterface, QueryRunner, Table } from "typeorm";

export class CreateUserTable1748000000000 implements MigrationInterface {

    public async up(queryRunner: QueryRunner): Promise<void> {
        await queryRunner.createTable(
            new Table({
                name: "user",
                columns: [
                    {
                        name: "id",
                        type: "int",
                        isPrimary: true,
                        isGenerated: true,
                        generationStrategy: "increment",
                    },
                    {
                        name: "firstName",
```

```
        type: "varchar",
        length: "100",
      },
      {
        name: "lastName",
        type: "varchar",
        length: "100",
      },
      {
        name: "createdAt",
        type: "timestamp",
        default: "now()",
      },
    ],
  })),
  true // ifNotExists – safe to run on non-empty DBs
);

public async down(queryRunner: QueryRunner): Promise<void> {
  await queryRunner.dropTable("user", true);
}
```

What is happening?

- `QueryRunner` wraps a single database connection and exposes both schema methods (`createTable` , `addColumn` , etc.) and raw query execution (`query()`).
- `new Table({...})` is TypeORM's schema description object — it mirrors what you would write in a `CREATE TABLE` statement but in a database-agnostic way.
- The second argument `true` to `createTable` adds an `IF NOT EXISTS` guard, making the migration idempotent if ever run manually against a database that already has the table.
- The `down()` method must *exactly* undo what `up()` did. For a table creation, the revert is a table drop.

Generating Migrations from Entities

Generated migrations are the far more common workflow. TypeORM compares your entity decorators against the current live schema and produces the minimal set of SQL to bring the database in line with the code. This is accurate, fast, and eliminates most hand-written SQL for routine schema changes.

Start with this entity:

src/entity/User.ts

```
import {
  Entity,
  PrimaryGeneratedColumn,
  Column,
  CreateDateColumn,
} from "typeorm";

@Entity()
export class User {
  @PrimaryGeneratedColumn()
  id!: number;

  @Column({ length: 100 })
  firstName!: string;

  @Column({ length: 100 })
  lastName!: string;

  @CreateDateColumn()
  createdAt!: Date;
}
```

Now suppose you need to add an `email` column and a `isActive` flag. Update the entity:

src/entity/User.ts

```
import {
  Entity,
  PrimaryGeneratedColumn,
  Column,
  CreateDateColumn,
} from "typeorm";
```



```
@Entity()
export class User {
  @PrimaryGeneratedColumn()
  id!: number;

  @Column({ length: 100 })
  firstName!: string;

  @Column({ length: 100 })
  lastName!: string;

  @Column({ unique: true, length: 255 }) // <-- added
  email!: string;

  @Column({ default: true }) // <-- added
  isActive!: boolean;

  @CreateDateColumn()
  createdAt!: Date;
}
```

Generate the migration:

```
npm run typeorm migration:generate src/migration/AddEmailAndIsActiveToUser
```

Output:

Migration /path/to/project/src/migration/1748000010000-AddEmailAndIsActiveToUser.ts has been generated successfully.

Inspect the generated file — TypeORM has written the exact SQL for you:

```
src/migration/1748000010000-AddEmailAndIsActiveToUser.ts
```

```
import { MigrationInterface, QueryRunner } from "typeorm";

export class AddEmailAndIsActiveToUser1748000010000 implements MigrationInterface {
  name = "AddEmailAndIsActiveToUser1748000010000";

  public async up(queryRunner: QueryRunner): Promise<void> {
    await queryRunner.query(
      `ALTER TABLE "user" ADD "email" character varying(255) NOT NULL`
    );
    await queryRunner.query(
```

```
`ALTER TABLE "user" ADD CONSTRAINT "UQ_user_email" UNIQUE ("email")`  
);  
await queryRunner.query(  
  `ALTER TABLE "user" ADD "isActive" boolean NOT NULL DEFAULT true`  
);  
}  
  
public async down(queryRunner: QueryRunner): Promise<void> {  
  await queryRunner.query(  
    `ALTER TABLE "user" DROP COLUMN "isActive"`  
  );  
  await queryRunner.query(  
    `ALTER TABLE "user" DROP CONSTRAINT "UQ_user_email"`  
  );  
  await queryRunner.query(  
    `ALTER TABLE "user" DROP COLUMN "email"`  
  );  
}  
}
```

Key Insight: Always review generated migration files before running them. TypeORM is highly accurate but occasionally generates a superfluous `ALTER` if default values differ between your local and remote database. Treat generated files as a solid first draft, not a guaranteed safe script.

Important: Generated migrations use raw SQL strings (`queryRunner.query()`) rather than the schema helper objects (`createTable` , etc.). Both approaches are valid. The raw SQL path is database-specific but easier to verify by reading; the schema helper path is database-agnostic but more verbose.

Running Migrations

The `migration:run` command executes all pending migrations — those whose class names do not yet appear in the `migrations` table — in ascending timestamp order.

```
npm run typeorm migration:run
```

Output:

```
query: SELECT * FROM "migrations" "migrations" ORDER BY "id" DESC
Running 2 migrations...
query: CREATE TABLE "user" ...
Migration CreateUserTable1748000000000 has been executed successfully.
query: ALTER TABLE "user" ADD "email" character varying(255) NOT NULL
query: ALTER TABLE "user" ADD CONSTRAINT "UQ_user_email" UNIQUE ("email")
query: ALTER TABLE "user" ADD "isActive" boolean NOT NULL DEFAULT true
Migration AddEmailAndIsActiveToUser1748000010000 has been executed successfully.
```

After the run, your database now contains two rows in the `migrations` table:

SQL – inspecting the migrations table

```
SELECT id, "timestamp", name FROM migrations ORDER BY id;
```

Output:

id	timestamp	name
1	1748000000000	CreateUserTable1748000000000
2	1748000010000	AddEmailAndIsActiveToUser1748000010000

Running `npm run typeorm migration:run` again produces no SQL — there is nothing pending:

Output:

```
No migrations are pending.
```

Running Migrations Programmatically

In some deployment pipelines (Docker entrypoints, AWS Lambda warm-ups) it is more reliable to run migrations from within the application's startup code rather than via the CLI. TypeORM's `DataSource` exposes a `runMigrations()` method for this purpose:

```
src/index.ts
```

```
import { AppDataSource } from "../data-source";

async function bootstrap(): Promise<void> {
  await AppDataSource.initialize();

  const pending = await AppDataSource.showMigrations();
  if (pending) {
    console.log("Running pending migrations...");
    await AppDataSource.runMigrations();
    console.log("Migrations complete.");
  }

  // Start your Express app here
}

bootstrap().catch(console.error);
```

Output:

```
Running pending migrations...
Migration AddEmailAndIsActiveToUser1748000010000 has been executed successfully.
Migrations complete.
```

Note: `showMigrations()` returns `true` if there are pending migrations, `false` otherwise. Use it as a guard to avoid the overhead of the migrations query on every restart when the schema is already current.

Reverting Migrations

`migration:revert` undoes the most recently applied migration by calling its `down()` method and removing its record from the `migrations` table. It only ever reverts *one migration at a time*.

```
npm run typeorm migration:revert
```

Output:

```
Reverting migration AddEmailAndIsActiveToUser1748000010000...  
query: ALTER TABLE "user" DROP COLUMN "isActive"  
query: ALTER TABLE "user" DROP CONSTRAINT "UQ_user_email"  
query: ALTER TABLE "user" DROP COLUMN "email"  
Migration AddEmailAndIsActiveToUser1748000010000 has been reverted successfully.
```

Run it again to revert the first migration:

```
npm run typeorm migration:revert
```

Output:

```
Reverting migration CreateUserTable1748000000000...  
query: DROP TABLE "user"  
Migration CreateUserTable1748000000000 has been reverted successfully.
```

Important: A `down()` that drops a column will destroy any data in that column. Before reverting a production migration, always take a database backup. In some teams, `down()` methods are intentionally left as a no-op or are made to archive data to a separate table rather than delete it outright.

Migration Best Practices

Never Edit a Migration That Has Already Run

Once a migration has been applied in any shared environment (staging, production), treat it as immutable. If you need to make a further change, create a new migration. Editing an existing one breaks the integrity of the `migrations` table — TypeORM matches by class name, so a rename or content edit will cause it to try to re-apply a migration that is already recorded.

Keep Migrations Small and Focused

One migration per logical change: adding a column, renaming a table, creating an index. Large migrations that bundle unrelated changes are harder to review and harder to safely revert. The timestamp-based ordering guarantees they run in the correct sequence.

Always Implement `down()`

A migration with an empty `down()` silently succeeds when reverting, leaving the database in an inconsistent state. Even if you never plan to revert, a correct `down()` is the safety net when something goes wrong.

Commit Migration Files to Version Control

Migration files in `src/migration/` must be committed alongside the entity changes that triggered them. A teammate who pulls your entity change without the corresponding migration will have a mismatch between their schema and the entity definition.

Production Workflow Summary

- Stage 1** Developer changes an entity, generates a migration.
 - Stage 2** Migration file is committed to Git and merged via pull request.
 - Stage 3** CI/CD pipeline deploys the new application code and runs `npm run typeorm migration:run` (or calls `runMigrations()` on startup).
 - Stage 4** On failure, the previous deploy is rolled back and `npm run typeorm migration:revert` restores the previous schema.
-

Summary

Concept	Key Point
<code>synchronize: true</code>	Auto-syncs schema on startup. Fast for development; unsafe for production due to potential data loss.
<code>migration:create</code>	Generates an empty migration file. Used for manual migrations (data backfills, complex DDL).
<code>migration:generate</code>	Diff's entity definitions against the live schema and writes the necessary SQL. Requires a running database connection.
<code>migration:run</code>	Executes all pending migrations in timestamp order; records each in the <code>migrations</code> table.
<code>migration:revert</code>	Calls <code>down()</code> of the last applied migration and removes its record. One migration per invocation.
<code>migrations</code> table	TypeORM's internal ledger of applied migrations. Never edit this manually.
<code>QueryRunner</code>	Gives direct access to the connection inside a migration; supports schema helpers and raw SQL execution.
<code>runMigrations()</code>	Programmatic equivalent of <code>migration:run</code> ; used in application startup code for automated deployment pipelines.
<code>-d</code> flag	Points the TypeORM CLI to the exported <code>DataSource</code> file. Required for all migration commands.

Interview Questions

1. What is the risk of running a TypeORM application in production with `synchronize: true` ?

Expected answer: TypeORM compares entity definitions to the live schema and issues DDL immediately on startup. A column removed from an entity or a changed type can silently drop or truncate data before any code runs. There is no review step and no `down()` to reverse the action.

2. What is the difference between `migration:create` and `migration:generate` ?

Expected answer: `migration:create` produces an empty file for the developer to fill in manually — useful for data migrations or complex DDL that cannot be inferred from decorators.

`migration:generate` connects to the database, diffs it against the current entity definitions, and writes the SQL automatically.

3. How does TypeORM know which migrations have already been applied?

Expected answer: It maintains a `migrations` table (configurable via `migrationsTableName`) that records the class name and timestamp of every successfully executed migration. On each run it compares the list of available migration classes against the table and only executes those not yet present.

4. What does the `down()` method of a migration do, and why should it always be implemented?

Expected answer: It undoes the schema change applied by `up()`, allowing `migration:revert` to restore the previous state. An empty `down()` means a revert call will silently succeed without actually changing the schema, leaving the database inconsistent with an earlier version of the code.

5. Why do migration files use a timestamp prefix in their filenames?

Expected answer: The timestamp (in milliseconds since epoch) provides a globally unique, sequential identifier. TypeORM uses it to determine execution order — migrations must be applied in the same order they were created to avoid dependency conflicts.

6. A colleague has edited an existing migration file that has already been applied to staging. What problem does this cause?

Expected answer: The `migrations` table stores the class name. Because the name matches,

TypeORM still considers the migration applied and will not re-run it. The edited SQL will never execute against staging. Any other environment that *has not* yet run that migration will run the edited version, causing the schema to diverge between environments.

7. How would you run migrations programmatically at application startup instead of via the CLI?

Expected answer: Call `await AppDataSource.runMigrations()` after

`AppDataSource.initialize()`. Optionally guard it with `await AppDataSource.showMigrations()` to skip the query when no migrations are pending.

8. What does the `-d` flag represent in TypeORM CLI commands such as `migration:generate`?

Expected answer: It specifies the path to the file that exports the `DataSource` instance. The CLI imports this file to get the database connection details, entity list, and migrations path needed to diff the schema and write or execute migration files.

Practical Assignment — Evolving a Schema Safely

Objective: Practise the full migration lifecycle: initial schema creation, an additive change, running both migrations, and reverting the most recent one.

Prerequisites: A running PostgreSQL instance and the project structure from Section 10/11.

1. In `src/data-source.ts`, ensure `synchronize` is `false` and `migrations` points to `["dist/migration/*.js"]`. Add the `typeorm` script to `package.json` as shown in this document.
2. Define a minimal `Product` entity in `src/entity/Product.ts` with `id` (auto-increment), `name` (varchar 200), and `price` (decimal). Add it to the `entities` array in your `DataSource`.
3. Run `npm run typeorm migration:generate src/migration/CreateProductTable` and inspect the generated file. Verify it contains a `CREATE TABLE` for `product` in `up()` and a `DROP TABLE` in `down()`.
4. Build the project (`npm run build`) and run the migration: `npm run typeorm migration:run`. Confirm the `product` table exists in your database.

5. Add a `stock` column (`integer` , default `0`) and a `createdAt` column (`@CreateDateColumn()`) to the `Product` entity. Generate a second migration named `AddStockAndCreatedAtToProduct` .
6. Build and run migrations again. Verify both columns appear in the `product` table.
7. Run `npm run typeorm migration:revert` . Confirm that `stock` and `createdAt` have been removed but the `product` table still exists. Check the `migrations` table — it should contain only the first migration.

Expected state after step 6:

Output:

id	timestamp	name
1	1748000020000	CreateProductTable1748000020000
2	1748000030000	AddStockAndCreatedAtToProduct1748000030000

Expected state after step 7:

Output:

id	timestamp	name
1	1748000020000	CreateProductTable1748000020000

Bonus Challenge: Write a migration *manually* (using `migration:create`) that seeds three rows into the `product` table using `queryRunner.query()` in `up()` , and deletes them by name in `down()` . Run and revert it to verify both directions work correctly.